



# TALK-TO-DATA (T2D)

Phase 9: Operate, adopt and improve

Daniel Brulé

[LinkedIn](#) - [GitHub](#)

Version 1.0 - June 2026

## Table of Contents

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| 1. Executive summary .....                                                       | 3  |
| 2. Phase overview .....                                                          | 4  |
| 2.1. Objective.....                                                              | 4  |
| 2.2. Scope of the phase .....                                                    | 4  |
| 2.3. What this phase does not do .....                                           | 5  |
| 2.4. Expected duration and level of effort .....                                 | 5  |
| 2.5. Main participants and decision owners.....                                  | 5  |
| 3. Operating lifecycle and scaling implications .....                            | 6  |
| 3.1. Possible Phase 9 operating routes .....                                     | 6  |
| 3.2. Minimum conditions for stable operation.....                                | 6  |
| 3.3. Run / fix / improve / expand model.....                                     | 7  |
| 3.4. Scaling implications .....                                                  | 7  |
| 3.5. How Phase 9 shapes longer-term maturity.....                                | 8  |
| 4. Operation, adoption and improvement activities overview .....                 | 9  |
| 4.1. Activity sequence .....                                                     | 9  |
| 4.2. Operating cadence .....                                                     | 9  |
| 4.3. Activity logic .....                                                        | 10 |
| 4.4. Practitioner note.....                                                      | 10 |
| 5. Core operating activities.....                                                | 11 |
| 5.1. Activate early-life support and operating cadence.....                      | 11 |
| 5.2. Monitor production usage, quality, trust, reliability and cost .....        | 12 |
| 5.3. Manage incidents, support requests and feedback-to-trace diagnosis.....     | 13 |
| 5.4. Maintain access lifecycle and protect the release boundary .....            | 14 |
| 5.5. Run live governance, residual-risk and decision-rights cadence .....        | 15 |
| 5.6. Maintain BAU service, data, semantic, metadata and documentation.....       | 16 |
| 5.7. Manage the run / fix / improve / expand backlog and capacity .....          | 17 |
| 5.8. Control service development, regression testing and version deployment..... | 18 |
| 5.9. Drive controlled adoption and responsible use within approved scope .....   | 19 |
| 5.10. Manage lifecycle, retirement and decommissioning readiness .....           | 20 |
| 6. Operating evidence decision pack .....                                        | 21 |
| 6.1. Operating evidence pack .....                                               | 21 |
| 6.2. Operating pack quality test .....                                           | 21 |
| 6.3. Output depth by operating maturity .....                                    | 22 |

|      |                                                                    |    |
|------|--------------------------------------------------------------------|----|
| 7.   | Exit criteria and handover .....                                   | 23 |
| 7.1. | Required transition outputs.....                                   | 23 |
| 7.2. | Handover to stable operation, improvement or lifecycle action..... | 23 |
| 8.   | Key risks and failure modes .....                                  | 24 |
| 8.1. | Practitioner note.....                                             | 24 |

# 1. Executive summary

Phase 9 operates the controlled production release after go-live and determines whether the Talk-to-Data capability remains useful, trusted, affordable and governed in real use.

This phase is not go-live approval. That decision was made in Phase 8. Phase 9 starts once production users are using the released capability inside an approved boundary. Its role is to stabilise the service, support users, monitor behaviour, manage incidents, govern change and decide whether the capability should run as-is, be fixed, improved, expanded, constrained, paused or eventually retired.

The hard part is not only keeping the system online. A Talk-to-Data capability can degrade because data changes, metrics evolve, metadata becomes stale, access rules change, model behaviour shifts, users move beyond scope, caveats are ignored or conversational usage drives cost beyond the expected envelope.

Phase 9 should therefore separate run, fix, improve and expand work. Operational issues should not automatically become roadmap items. User demand should not automatically justify expansion. Improvements should be versioned, tested, monitored, governed and communicated. Expansion should be earned through production evidence, not triggered by enthusiasm or high usage alone.

A narrow stable release is a valid success. Phase 9 should track whether the capability is delivering value, being used responsibly, staying within risk appetite, remaining affordable and operating sustainably. Where that is no longer true, the right decision may be to constrain usage, pause a scope area, roll back a version, retire part of the service or prepare for decommissioning.

The main output is operating evidence: monitoring and support findings, feedback-to-trace analysis, governance and change records, regression evidence, backlog and capacity view, adoption findings, cost and risk reviews, and lifecycle records.

## 2. Phase overview

Phase 9 operates the controlled production release after go-live. It starts from the Phase 8 release decision, approved production boundary, known limitations, residual-risk acceptance, support model, monitoring setup, runbooks, onboarding material, backlog and Phase 9 handover pack.

The purpose of this phase is not to approve go-live again. It is to run the released capability responsibly in real use: support users, monitor behaviour, manage incidents, govern change, protect the release boundary and adapt the service as evidence emerges.

Phase 9 should also recognise that Talk-to-Data sits at the end of a wider analytical flow. Changes in ingestion, data modelling, semantic definitions, metadata, access rules, reporting logic or upstream platforms can affect answers even when the T2D application itself has not changed. Operation therefore requires regular interaction with upstream owners so material changes are detected early, assessed, tested and deployed safely where needed.

### 2.1. Objective

The objective of Phase 9 is to operate, support and improve the controlled production release while keeping it useful, trusted, affordable and governed.

The phase should monitor whether users are asking supported questions, interpreting answers correctly, understanding caveats, escalating issues appropriately and staying within the intended usage pattern. It should also maintain the production operating controls required for support, incident handling, access lifecycle, monitoring, feedback-to-trace analysis, regression testing, change control, cost review, governance, documentation and controlled adoption.

Where the service needs to change, Phase 9 should separate run, fix, improve and expand work. Defects should be handled as fixes. Better answer formats, clearer caveats, lower latency or lower cost may be improvements. New users, domains, question types, integrations or higher-risk use cases should be treated as expansion and require stronger evidence, capacity and governance.

Phase 9 also keeps lifecycle options visible. A T2D service may continue, improve, expand, narrow, pause, be absorbed into another product, retire or eventually decommission. This is not expected immediately, but a service that cannot be stopped cleanly is not fully controlled..

### 2.2. Scope of the phase

Phase 9 covers the live operation of the released T2D capability within the boundary approved in Phase 8. It keeps the service usable, trusted and controlled in real conditions: support, monitoring, incidents, access lifecycle, documentation, governance, change control, adoption and improvement.

The phase should also protect the service from uncontrolled expansion. Adjacent needs should be captured, but not automatically absorbed into the live capability. Requests for new users, domains, question types, integrations or higher-risk use should follow the expansion route.

The scope should remain proportionate to release exposure. A narrow internal release may use a lightweight but explicit operating cadence. A broader, decision-critical or regulated release requires stronger evidence, more formal governance, tighter change control and more frequent review.

### 2.3. What this phase does not do

Phase 9 is not broad rollout by default. More users, domains, question types or integrations should not be added simply because the first production release is live.

It is not a feature factory. User requests should be triaged into run, fix, improve or expand categories, with different evidence, ownership and approval routes.

It is not permanent firefighting. Repeated incidents, unsupported questions, high support load, cost spikes, misuse or trust degradation should trigger root-cause analysis and governance action, not informal workarounds.

It is not a replacement for upstream governance. Phase 9 should detect and manage the T2D impact of upstream changes, but ownership of data, semantic, metadata, access and platform assets remains with the relevant owners.

It is also not indefinite operation at any cost. If the service no longer delivers enough value, cannot be supported sustainably, becomes too costly, depends on obsolete foundations or is replaced by a better route, Phase 9 should support controlled retirement or decommissioning.

### 2.4. Expected duration and level of effort

Phase 9 begins immediately after controlled production release. It usually starts with early-life support, then moves into stable operation once usage, incidents, cost, access, support load and trust signals are understood.

Unlike earlier phases, Phase 9 does not have a fixed end date. It is the operating lifecycle of the capability. Its effort should be reviewed regularly and adjusted based on usage, risk, cost, support burden, adoption, roadmap demand and upstream change activity.

### 2.5. Main participants and decision owners

Phase 9 requires business, product, operating, support, data, semantic, metadata, platform, AI, security, risk and adoption ownership.

The important point is that T2D operation should not sit only with the application team. Because the service depends on upstream data, semantic, metadata, access and platform flows, Phase 9 governance must include the owners who can affect live answer behaviour before users see the impact.

## 3. Operating lifecycle and scaling implications

Phase 9 does not end with a single approval decision. It creates the operating evidence and governance rhythm needed to keep the released T2D capability controlled over time. The service may continue as-is, need fixes, improve incrementally, expand selectively, narrow its scope, return to an earlier phase, pause, retire part of the capability or eventually be decommissioned.

The key discipline is to avoid treating production usage as proof of readiness for scale. Live operation creates evidence, but that evidence must be interpreted carefully. High usage may show value, but it may also reveal over-reliance, unsupported demand, rising cost or support fragility. Low usage may show weak adoption, but it may also reflect poor onboarding, limited workflow fit, low trust or unclear ownership.

### 3.1. Possible Phase 9 operating routes

Phase 9 should maintain a live view of the appropriate operating route for the capability. These routes are not failure labels; they are operating controls.

| Route                          | Meaning                                                                                            |
|--------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Run</b>                     | Continue operating within the approved boundary.                                                   |
| <b>Fix</b>                     | Correct a defect, incident, misleading answer, access issue, broken trace or failed control.       |
| <b>Improve</b>                 | Improve current-boundary usability, explanation, latency, cost, supportability or answer quality.  |
| <b>Expand</b>                  | Assess new users, domains, question types, integrations or higher-risk use cases.                  |
| <b>Constrain</b>               | Limit affected users, questions, data, answer types or usage patterns.                             |
| <b>Return to earlier phase</b> | Route structural data, semantic, architecture, validation or pilot issues back to the right phase. |
| <b>Pause</b>                   | Temporarily stop affected scope while risk, quality or control issues are resolved.                |
| <b>Retire / decommission</b>   | Remove part or all of the service when continued operation is no longer justified.                 |

### 3.2. Minimum conditions for stable operation

A controlled release becomes stable operation only when the service can be run without exceptional project-team effort. The minimum conditions are:

| Condition                | What it means                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------------|
| <b>Boundary control</b>  | Users, data, questions, answer types and usage limits remain within the approved scope.            |
| <b>Supportability</b>    | Support teams can diagnose issues using runbooks, traces, logs and escalation routes.              |
| <b>Trust calibration</b> | Users understand when to rely on answers, when to challenge them and when to escalate.             |
| <b>Quality control</b>   | Answer-quality issues, refusals, unsupported questions and regressions are monitored and acted on. |

|                           |                                                                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Cost control</b>       | Usage, model calls, query cost, logging and support effort remain within the expected envelope.       |
| <b>Change control</b>     | Model, prompt, data, metadata, semantic, access and UI changes are versioned, tested and governed.    |
| <b>Upstream awareness</b> | Data, semantic, metadata, access and platform changes are detected early enough to assess T2D impact. |
| <b>Ownership</b>          | Product, operating, data, semantic, platform, security and governance owners remain engaged.          |

Stable operation does not mean the service is perfect. It means the team can see problems, diagnose them, decide who owns them and act before trust, cost or risk become uncontrolled.

### 3.3. Run / fix / improve / expand model

Phase 9 should separate four types of work.

| Work type      | Typical trigger                                                                               | Response                                                             |
|----------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Run</b>     | Normal support, monitoring, access administration and operating reviews.                      | Handle through standard operating cadence.                           |
| <b>Fix</b>     | Defect, incident, misleading answer, access issue, broken trace or failed control.            | Prioritise, correct, retest and communicate where behaviour changes. |
| <b>Improve</b> | Current scope works, but usability, explanation, latency, cost or supportability can improve. | Prioritise through backlog and release through controlled change.    |
| <b>Expand</b>  | Users request new scope, domains, questions, integrations or higher-risk use cases.           | Treat as new evidence-backed demand, not automatic product growth.   |

This distinction matters because each type of work needs different evidence and decision rights. A defect should not become a roadmap debate. An expansion request should not be hidden inside a minor change. A support workaround should not become the permanent operating model.

### 3.4. Scaling implications

Scaling should be earned through production evidence. Before expanding users, domains, questions or integrations, Phase 9 should show that the current release is valuable, trusted, supportable, affordable and controlled.

Expansion should be considered only when the data foundation, semantic definitions, metadata, access controls, monitoring, support capacity, regression tests, funding and governance route are strong enough for the proposed increase in exposure.

Where live usage reveals adjacent needs, the Phase 9 team should capture and route them. Some may belong inside the current T2D capability. Others may require a separate T2D use case, dashboard, data-quality fix, semantic-model change, analyst workflow, process redesign or different product capability. The team may be well placed to evidence the need, but it should not automatically absorb it into the live service.



### 3.5. How Phase 9 shapes longer-term maturity

Phase 9 is where T2D becomes a managed capability rather than a delivered project. It shapes maturity in five areas:

| Area                      | Maturity implication                                                                         |
|---------------------------|----------------------------------------------------------------------------------------------|
| <b>Product</b>            | Clearer view of what users value, misuse, request and trust.                                 |
| <b>Data and semantics</b> | Stronger feedback loop on definitions, caveats, metadata, quality and upstream change.       |
| <b>Platform</b>           | Better understanding of model routing, latency, cost, observability and deployment patterns. |
| <b>Governance</b>         | Clearer decision rights for risk, access, change, expansion, pause and retirement.           |
| <b>Operating model</b>    | Better sizing of support, engineering, data, semantic, product and adoption capacity.        |

The aim is not to scale by default. The aim is to operate well, learn from evidence and evolve deliberately.

## 4. Operation, adoption and improvement activities overview

Phase 9 should be run as an operating loop, not as a one-off post-launch review. The activities below create the rhythm needed to support users, monitor behaviour, govern change, improve safely and keep the service within its approved boundary.

The sequence is directional. In practice, several activities run in parallel: support, monitoring, access control, governance, upstream change review and backlog management should operate continuously once the service is live.

### 4.1. Activity sequence

| #  | Activity                                                                  | Purpose                                                                                                              |
|----|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 1  | Activate early-life support and operating cadence                         | Move from release handover into live support, review rhythm, escalation and ownership.                               |
| 2  | Monitor production usage, quality, trust, reliability and cost            | Build the evidence base for operating decisions and safe improvement.                                                |
| 3  | Manage incidents, support requests and feedback-to-trace diagnosis        | Link user issues and feedback to traces, root causes and accountable owners.                                         |
| 4  | Maintain access lifecycle and protect the release boundary                | Control users, roles, permissions, usage limits and scope drift.                                                     |
| 5  | Run live governance, residual-risk and decision-rights cadence            | Keep risk, change, cost, adoption and boundary decisions accountable.                                                |
| 6  | Maintain BAU service, data, semantic, metadata and documentation currency | Keep the live service and its upstream analytical foundations current, tested and aligned with production behaviour. |
| 7  | Manage the run / fix / improve / expand backlog and capacity              | Separate operational work, defects, improvements and expansion demand before prioritisation.                         |
| 8  | Control service development, regression testing and version deployment    | Build, test and release approved fixes, optimisations and current-boundary improvements safely.                      |
| 9  | Drive controlled adoption and responsible use within approved scope       | Encourage appropriate usage without creating uncontrolled demand.                                                    |
| 10 | Manage lifecycle, retirement and decommissioning readiness                | Ensure the service can be narrowed, paused, retired or eventually decommissioned cleanly.                            |

### 4.2. Operating cadence

Phase 9 should start with early-life support and then move into stable operation once the service is predictable enough to run without exceptional project-team effort.

A typical cadence may include daily checks during the first days after release, weekly operating reviews during early-life support, and regular governance reviews once the service stabilises. The cadence should cover usage, incidents, unsupported questions, access changes, cost, latency, feedback, upstream changes, regression results, backlog pressure and adoption signals. The cadence should remain proportionate. A narrow internal release may need a

lightweight rhythm. A decision-critical, regulated or broad-user release requires stronger monitoring, clearer escalation and more formal governance.

### 4.3. Activity logic

The activities follow a simple operating logic:

1. Stabilise the live release.
2. Observe real production behaviour.
3. Diagnose issues using evidence, not anecdotes.
4. Protect the approved boundary.
5. Govern risk, cost, change and ownership.
6. Maintain upstream and downstream currency.
7. Prioritise work based on value, evidence and capacity.
8. Build, test and release approved changes safely.
9. Grow adoption only where use remains responsible and supportable.
10. Keep lifecycle options available, including pause, retirement and decommissioning.

The aim is not to make the service static but to let it evolve without losing control.

### 4.4. Practitioner note

The main danger in Phase 9 is silent expansion. A live T2D service will attract new users, new questions and new expectations. It will also reveal unmet needs that may belong in dashboards, semantic models, data-quality remediation, analyst workflows, another T2D use case or a different product. The Phase 9 team should capture and evidence that demand, but not automatically absorb it into the live service.

A stable narrow release is a valid success. The service should grow only when production evidence shows that value, trust, cost, support, data readiness, semantic maturity and governance can grow with it.

## 5. Core operating activities

### 5.1. Activate early-life support and operating cadence

**Purpose:** Move from controlled release handover into live operation with clear support, monitoring, escalation, ownership and review cadence.

**Core activities:**

- Confirm that the Phase 8 release boundary, known limitations, support model, monitoring setup, runbooks, escalation routes, residual risks and Phase 9 handover pack are understood by the operating team.
- Confirm who monitors the live service, handles questions, manages incidents, owns access changes, reviews quality / cost, and can pause, narrow or roll back the service if needed.
- Activate early-life support for the first days and weeks after release, including the review cadence, support coverage, escalation path and issue-triage route.
- Review production signals regularly, including usage, incidents, unsupported questions, answer-quality concerns, refusals, latency, cost, access issues, support load and feedback.
- Establish a regular upstream change check with data, semantic, metadata, access, reporting and platform owners so changes are identified before they affect live T2D.
- Confirm the route for moving from early-life support into stable operation once usage, incidents, trust, cost and support load are predictable enough.

**Output:**

- Activated operating cadence with named owners, review rhythm and escalation route.
- Early-life plan covering monitoring, support coverage, issue triage and communication.
- Confirmed pause, rollback and scope-narrowing authority.
- First operating review schedule.
- Early-life issue and signal log.
- Upstream change review route.

**Red flags:**

- Phase 9 starts with informal support from the build team rather than operating owners.
- Users are live but support, monitoring, escalation or incident routes are unclear.
- Nobody is empowered to pause, narrow or roll back the service if early issues appear.
- Upstream data, semantic, metadata or access changes are discovered only after users report answer changes.
- Early-life support turns into permanent firefighting instead of a controlled transition.

**Practitioner note:**

A controlled release is not operated because it has gone live. It is operated when the team can see what is happening, understand who owns each signal, respond before trust is damaged and stop or narrow usage if the service behaves outside its approved boundary.

## 5.2. Monitor production usage, quality, trust, reliability and cost

**Purpose:** Monitor how the live T2D service behaves in real production use, so issues, risks, demand patterns and improvement opportunities are detected early.

### **Core activities:**

- Track usage by user group, question type, domain, answer type and interaction volume.
- Monitor supported, unsupported and ambiguous questions against the approved boundary.
- Review answer-quality signals: failed answers, weak grounding, missing caveats, refusals and clarifications.
- Monitor trust signals: over-reliance, ignored caveats, repeated rephrasing or use outside workflow.
- Track reliability and performance: availability, errors, latency, timeouts and failed tool calls.
- Review cost drivers: model calls, follow-up depth, query execution, logging and support effort.
- Compare live signals against Phase 8 assumptions, thresholds and accepted residual risks.
- Route trends into run, fix, improve, expand, constrain or governance action.

### **Output:**

- Production monitoring dashboard or signal pack.
- Usage, quality, trust, reliability and cost summary.
- Unsupported-question and out-of-scope demand log.
- High-cost, high-latency and high-risk interaction review.
- Threshold breach and escalation record.
- Issues routed into the run / fix / improve / expand backlog.

### **Red flags:**

- Monitoring focuses only on uptime.
- High usage is treated as success without checking trust, cost or supportability.
- Unsupported questions increase without action.
- Follow-up conversations drive cost without review.
- Signals are collected but not owned.

### **Practitioner note:**

For T2D, production monitoring is not just technical observability. Some signals can be captured automatically, such as usage, latency, errors, refusals, cost and failed tool calls. Others require active review through user feedback, support conversations, answer-quality checks or interviews. Over-trust, caveat misunderstanding, inappropriate reliance and workflow misuse will rarely be visible from logs alone.

### 5.3. Manage incidents, support requests and feedback-to-trace diagnosis

**Purpose:** Convert incidents, support requests and user feedback into traceable evidence so issues can be diagnosed, owned and acted on.

**Core activities:**

- Capture incidents, support requests and feedback, and link each issue to the user group, question type, answer, trace, data asset and model or prompt version.
- Classify the root cause: user guidance, data, semantics, metadata, access, orchestration, model behaviour, UI, cost, latency or support process.
- Decide whether the fix belongs in the T2D layer or upstream in data, modelling, semantics, metadata or access controls.
- Route issues into run, fix, improve, expand, constrain or governance action, with the right owner.
- Identify recurring unsupported questions, misunderstood answers or user workarounds.
- Communicate fixes, limitations or behaviour changes to users and support teams.
- Retain evidence for audit, quality review, regression testing and backlog prioritisation.

**Output:**

- Incident, support and feedback log.
- Feedback-to-trace register.
- Root-cause classification summary.
- Escalation and ownership record.
- User communication or support note where required.
- Items routed into run / fix / improve / expand backlog.

**Red flags:**

- Feedback is reviewed without linking it to traces or outputs.
- Support teams repeatedly explain the same limitation manually.
- User complaints are treated as opinion rather than diagnostic evidence.
- Incidents are fixed without checking whether similar questions are affected.
- Recurring unsupported questions become informal scope expansion.

**Practitioner note:**

A user saying “the answer is wrong” is rarely enough to diagnose the issue. The team needs the trace: what was asked, what was retrieved, what query ran, what data was returned, what model version answered, what caveats were shown and how the user interpreted the result. Without this link, support becomes guesswork.

Some issues can be fixed directly in the T2D layer, such as answer wording, prompt behaviour, clarification logic or UI. Others depend on upstream data, modelling, semantic, metadata or access changes. Especially in early production, fixes may take longer because the correct owner sits outside the T2D team. The operating discipline is to balance speed with correctness.

## 5.4. Maintain access lifecycle and protect the release boundary

**Purpose:** Ensure users, permissions, data exposure and usage remain inside the approved production boundary.

**Core activities:**

- Maintain onboarding, offboarding, role changes and permission reviews for production users.
- Confirm that users can only access approved domains, data assets, metrics, dimensions and answer types.
- Monitor whether user questions, follow-ups, exports or workflows drift beyond the release boundary.
- Review access exceptions, denied requests, exposure concerns and role-change delays.
- Update user guidance when unsupported or restricted use patterns recur.
- Route requests for new users, domains, questions or integrations through expansion governance.
- Constrain, pause or remove access where usage creates risk, cost pressure or trust concerns.

**Output:**

- Access lifecycle log.
- Release boundary and usage exception log.
- Approved user and role list.
- Access issue and exposure review.
- Boundary-change requests routed to governance.
- Updated user guidance where needed.

**Red flags:**

- Users are added informally because the service is “already live”.
- Access changes are handled faster than access evidence can be checked.
- Unsupported questions become accepted through repeated manual support.
- Users export or reuse answers outside the intended workflow.
- Nobody owns the decision to constrain or remove access.

**Practitioner note:**

The release boundary is not a document produced in Phase 8 and forgotten. It must be actively operated. In T2D, scope can expand silently through new users, role changes, follow-up questions, exports, copied answers or informal support. Access control should therefore cover not only who can log in, but what they are allowed to ask, see, trust and reuse.

During early adoption, especially with small cohorts, temporary shortcuts may help users get started. That can be acceptable if they are visible, time-bound and owned. The risk is when shortcuts become the operating model. Any manual access workaround, support workaround or boundary exception should be logged, reviewed and either formalised, fixed or removed once adoption stabilises.

## 5.5. Run live governance, residual-risk and decision-rights cadence

**Purpose:** Keep the live T2D service accountable after release by reviewing risk, cost, change, adoption, boundary and lifecycle decisions with the right owners.

**Core activities:**

- Run regular governance reviews covering service health, incidents, risks, cost, adoption, change and boundary pressure.
- Review residual risks accepted at release and confirm whether they remain acceptable in real use.
- Confirm who can approve fixes, improvements, model changes, scope changes, expansion, constraints, pause or rollback.
- Include upstream owners for data, modelling, semantics, metadata, access, reporting and platform changes.
- Review upcoming upstream changes and assess their likely impact on T2D behaviour before users are affected.
- Escalate structural issues to the right phase or owner rather than absorbing them as operational noise.
- Maintain a decision log for material risk, change, cost, adoption, expansion and lifecycle decisions.

**Output:**

- Live governance cadence and attendance record.
- Residual-risk review log.
- Decision-rights matrix.
- Material change and boundary decision log.
- Upstream change impact record.
- Escalation record for issues requiring earlier-phase remediation.

**Red flags:**

- Governance reviews happen without data, semantic, access or platform owners.
- Residual risks are accepted once and never reviewed again.
- Material changes are approved informally because the service is already live.
- Upstream changes are detected only after answer behaviour changes.
- Expansion decisions are driven by demand rather than evidence, capacity and control.

**Practitioner note:**

Governance after go-live should not become theatre. Its role is to keep the live service within the release boundary, make risk and change decisions explicit, and ensure upstream changes are understood before they affect users. A T2D service is downstream of data, semantic, metadata, access and platform flows; governance must include the people who can change its behaviour, with more frequent reviews during early adoption when usage patterns, risks and support needs are still stabilising.



## 5.6. Maintain BAU service, data, semantic, metadata and documentation

**Purpose:** Keep the live T2D service and its upstream data, semantic, metadata and documentation foundations current, tested and aligned with production behaviour as part of business-as-usual operation.

### **Core activities:**

- Run automated checks and alerts on source freshness, schema, format, volume, quality, reconciliation, metadata availability and access controls.
- Run lightweight BAU checks on authentication, UI flow, feedback capture, core supported questions, links, exports and monitoring where applicable.
- Assess whether upstream data, modelling, semantic, metadata, access or reporting changes may affect live T2D answers, workflows, caveats or trust.
- Use human review where judgement is required, especially for semantic changes, caveats, anomalies, interpretation risk or material business change.
- Update metadata, examples, glossary terms, caveats, known limitations, user guidance and support material where needed, and communicate user-facing changes.
- Run regular BAU regression checks where service, upstream or foundation changes may affect supported questions, retrieval, grounding or answer behaviour.

### **Output:**

- Automated data, metadata, access and service check results.
- Data, semantic, metadata and documentation currency log.
- Upstream change impact assessment.
- Updated caveats, known limitations, user guidance and support material.
- Issues routed to data, semantic, metadata, access, platform or service owners.
- BAU regression check results or trigger record.

### **Red flags:**

- Automated checks pass but semantic, caveat or user-facing behaviour changes are not reviewed.
- Metric, caveat or upstream changes are made but not reflected in T2D.
- Users discover data, definition or service changes before the operating team does.
- Known limitations are outdated or inconsistent across guidance and support material.
- Documentation is treated as static after release.

### **Practitioner note:**

Automation should do as much routine control work as possible: freshness, schema, quality, reconciliation, access, metadata and basic service checks. But automation will not reliably detect whether a definition is still business-appropriate, whether a caveat is understandable, or whether users are interpreting answers correctly. Those signals come from regular interaction with users: structured reviews such as focus groups are useful, but informal check-ins with representatives of the main user groups are often just as effective.

## 5.7. Manage the run / fix / improve / expand backlog and capacity

**Purpose:** Convert operating signals, incidents, BAU checks and user demand into a prioritised backlog, without mixing support work, defects, optimisation, improvement and expansion.

**Core activities:**

- Consolidate items from monitoring, incidents, feedback, BAU checks, governance reviews and user requests.
- Classify each item as run, fix, optimise, improve, expand, constrain or defer.
- Confirm whether the work belongs to the T2D team or to upstream data, semantic, metadata, access or platform owners.
- Prioritise using value, risk, trust, cost, urgency, support load, evidence and available capacity.
- Separate current-boundary improvements from expansion to new users, domains, questions, integrations or higher-risk use cases.
- Capture adjacent needs and route them to the right path: current T2D, new T2D use case, dashboard, data fix, semantic work, workflow change or another product.
- Size the capacity needed across product, support, engineering, data, semantic, AI, platform and governance owners.

**Output:**

- Run / fix / optimise / improve / expand backlog.
- Prioritisation and capacity view.
- Ownership and upstream dependency record.
- Adjacent-needs routing log.
- Expansion candidates separated from current-release work.
- Items approved for controlled change and deployment.

**Red flags:**

- All user requests are treated as product improvements.
- Expansion items are hidden inside minor backlog changes.
- Upstream issues are assigned to the T2D team by default.
- Support work consumes all capacity and blocks fixes or improvements.
- The backlog grows without funding, ownership or prioritisation.

**Practitioner note:**

A live T2D service will generate more demand than the team should absorb. Some items are defects, some are BAU work, some are useful improvements, and some are evidence of a different need altogether. The discipline is to classify before committing. Otherwise the service becomes a feature factory, a helpdesk, or a workaround layer for upstream issues that should be fixed elsewhere.

## 5.8. Control service development, regression testing and version deployment

**Purpose:** Build, test and release fixes, optimisations and current-boundary improvements safely, with clear versioning, regression evidence, communication and rollback routes.

**Core activities:**

- Confirm that each change is approved through the backlog, governance or hot-fix route.
- Develop approved changes through controlled code, prompt, model, configuration, UI or monitoring workflows.
- Classify the change: code, prompt, model, routing, retrieval, validation, UI, configuration, monitoring or dependency.
- Review approved model options, including smaller or specialist models, where they may improve quality, latency, cost or control.
- Assess impact on answers, access, cost, latency, reliability, user, support and auditability.
- Run regression tests on supported questions, edge cases, access scenarios, refusals, caveats and known failure modes.
- Version changed components and link them to test, release notes and rollback route.
- Deploy through controlled release or hot-fix process, then monitor post-release behaviour.
- Communicate user-facing behaviour changes to users and support teams.

**Output:**

- Change classification and impact assessment.
- Development and review record.
- Regression and evaluation evidence.
- Version record for changed components.
- Release note or support communication.
- Deployment, monitoring and rollback record.
- Post-release behaviour review.

**Red flags:**

- Changes are developed without clear approval, ownership or version control.
- Prompt, model or routing changes are treated as minor configuration updates.
- Smaller models are adopted for cost reasons without quality evidence.
- Fixes are released without testing supported questions and known edge cases.
- Users notice behaviour changes before support teams are informed.
- Rollback is not possible because versions or dependencies were not recorded.

**Practitioner note:**

In T2D, a small technical change can alter user trust. A prompt edit, model substitution, routing change, retrieval tweak, validation rule or UI wording change can affect what users ask, what the system retrieves, what it refuses and what users believe. Treat behavioural change as a release, not housekeeping.

## 5.9. Drive controlled adoption and responsible use within approved scope

**Purpose:** Encourage appropriate use of the live T2D service without creating uncontrolled demand, over-reliance or support burden.

**Core activities:**

- Onboard approved users with clear guidance on supported questions, caveats, limits and escalation routes.
- Reinforce responsible use: when to rely on answers, verify them, challenge them or escalate.
- Monitor adoption by user group, workflow fit, repeat usage, trust signals and support load.
- Use feedback and user check-ins to identify friction, misunderstanding, unmet needs and responsible-use issues.
- Communicate fixes, behaviour changes, known limitations and scope reminders in plain language.
- Use representative users to test understanding, surface adoption friction and validate responsible-use guidance
- Route requests for new users, domains, questions or workflows through the backlog and expansion route.

**Output:**

- User onboarding and responsible-use record.
- Adoption, usage and workflow-fit summary.
- User feedback and check-in notes.
- Guidance or communication updates.
- Scope reminder or limitation communication where needed.
- Expansion or adjacent-needs requests routed to the right path.

**Red flags:**

- Adoption is measured only by usage volume.
- Users rely on answers without understanding caveats or verification needs.
- Support teams become the default route for every analytical question.
- Representative users create informal shortcuts, scope expansion or unsupported workarounds.
- High demand leads to new users or questions being added without evidence, capacity or governance.

**Practitioner note:**

Adoption is not the same as consumption. More usage is valuable only when the right users rely on the service in the right workflow, within the approved boundary and with the right level of trust. Strong demand from representative users is a positive signal, but it should create evidence for controlled improvement or expansion, not automatic scope growth.

## 5.10. Manage lifecycle, retirement and decommissioning readiness

**Purpose:** Ensure the live T2D service can be constrained, paused, retired or eventually decommissioned cleanly when continued operation is no longer justified.

**Core activities:**

- Define the triggers for constraint, pause, retirement or decommissioning: low value, low usage, high cost, recurring risk, weak ownership, replacement or obsolete foundations.
- Review whether parts of the service are still justified: users, domains, questions, metrics, integrations, models, workflows and support routes.
- Confirm who can approve lifecycle decisions and what evidence is required before narrowing, retiring or decommissioning.
- Prepare user, support, audit, access, data-retention, documentation, backlog and cost actions for any retired scope.
- Ensure retired or paused scope is reflected in user guidance, known limitations, access rules, monitoring and support material.
- Capture lessons from retirement or decommissioning to inform future T2D releases or replacement capabilities.

**Output:**

- Lifecycle review record.
- Constraint, pause, retirement or decommissioning trigger log.
- Scope retirement or decommissioning decision record.
- User, support and stakeholder communication.
- Access, monitoring, documentation and backlog closure record.
- Lessons learned for future releases or replacement services.

**Red flags:**

- The service continues because nobody wants to own the stop decision.
- Low usage, high cost or weak value is explained away without review.
- Scope is quietly disabled without user communication or evidence.
- Retired metrics, questions or users remain visible in guidance, access or monitoring.
- Open incidents, backlog items or audit evidence are not closed cleanly.

**Practitioner note:**

Every production service has a lifecycle. Decommissioning is not expected immediately after release, but it should be possible one day. A mature T2D capability should be able to continue, improve, expand, narrow, pause, retire or close cleanly.

Lifecycle decisions should not rely on usage volume alone. Low usage may indicate weak value, poor adoption or an obsolete service, but it may also reflect a critical capability used only during specific planning, reporting, incident or decision cycles. The question is not only “how often is it used?”, but whether the value, risk, cost, ownership and support model still justify keeping it live.

## 6. Operating evidence decision pack

Phase 9 should produce evidence that the live T2D service is being operated, monitored, governed and improved in a controlled way. The outputs should not become a heavy reporting pack. They should help the team run the service, diagnose issues, control change, manage adoption and prepare for longer-term lifecycle decisions.

### 6.1. Operating evidence pack

The main Phase 9 output is an operating evidence pack. It should consolidate the evidence needed to understand how the live service is performing and whether it remains within the approved release boundary.

The pack should include:

- operating cadence, support route and escalation records;
- production monitoring summary covering usage, quality, trust, reliability, access, cost and support load;
- incident, support and feedback-to-trace records;
- access lifecycle and release-boundary exception logs;
- governance, residual-risk and decision records;
- BAU data, semantic, metadata, documentation and service-check evidence;
- run / fix / improve / expand backlog and capacity view;
- change, regression, version and release records;
- adoption, responsible-use and representative-user feedback evidence;
- lifecycle, pause, retirement or decommissioning records where relevant.

The evidence pack should remain proportionate. A narrow release may only need a concise operating summary, issue log and change record. A broader, decision-critical or regulated release will need stronger traceability, more formal governance records and clearer evidence of risk, cost and change control.

### 6.2. Operating pack quality test

| Test                        | Question                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------|
| <b>Operating clarity</b>    | Can the team see how the live service is performing?                                 |
| <b>Boundary control</b>     | Is it clear whether users, questions, data and usage remain inside scope?            |
| <b>Diagnostic value</b>     | Can incidents and feedback be linked to traces, root causes and owners?              |
| <b>Governance</b>           | Are material risk, cost, change and lifecycle decisions recorded?                    |
| <b>Upstream awareness</b>   | Are upstream data, semantic, metadata, access and platform changes visible?          |
| <b>Change control</b>       | Are fixes, optimisations and improvements versioned, tested and releasable?          |
| <b>Adoption control</b>     | Is usage growing responsibly rather than silently expanding scope?                   |
| <b>Lifecycle discipline</b> | Can the service be constrained, paused, retired or decommissioned cleanly if needed? |

### 6.3. Output depth by operating maturity

| Output                    | Narrow release                             | Production / higher-risk release                             |
|---------------------------|--------------------------------------------|--------------------------------------------------------------|
| <b>Operating evidence</b> | Short operating summary                    | Formal operating evidence pack                               |
| <b>Monitoring</b>         | Basic usage, cost, error and feedback view | Quality, trust, access, cost, latency and incident dashboard |
| <b>Feedback diagnosis</b> | Simple issue log                           | Feedback-to-trace register with root-cause classification    |
| <b>Governance</b>         | Product and technical review notes         | Cross-functional governance and residual-risk records        |
| <b>BAU checks</b>         | Key data and service checks                | Automated checks with evidence and alert routes              |
| <b>Change control</b>     | Version and test record                    | Controlled release, regression and rollback evidence         |
| <b>Backlog</b>            | Combined prioritised backlog               | Run / fix / improve / expand backlog with capacity view      |
| <b>Adoption</b>           | User feedback notes                        | Responsible-use and representative-user evidence             |
| <b>Lifecycle</b>          | Basic pause route                          | Retirement and decommissioning decision records              |

## 7. Exit criteria and handover

Phase 9 does not have a fixed end point in the same way as earlier delivery phases. It is the operating lifecycle of the live T2D capability. However, it should still have clear transition points: from early-life support to stable operation, from stable operation to controlled improvement, from current release to expansion, or from continued operation to pause, retirement or decommissioning.

The key question is not whether Phase 9 is “complete”. The question is whether the service is being operated under control, with enough evidence, ownership and governance to decide what should happen next.

### 7.1. Required transition outputs

Before reducing early-life support, approving a new release cycle, expanding the service or moving towards retirement, the team should have the operating evidence decision pack described in Section 6, completed to the standard required by release exposure, operating maturity and risk.

The transition decision should confirm the relevant route: stable operation, controlled improvement, expansion assessment, earlier-phase remediation, constraint, pause, retirement or decommissioning. It should also identify the live boundary, operating evidence, unresolved risks, owners, capacity, next actions and communication needs..

### 7.2. Handover to stable operation, improvement or lifecycle action

Phase 9 can hand over into several routes.

| Route                                | Handover requirement                                                                                              |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Stable operation</b>              | Support, monitoring, access, governance and BAU checks are working without exceptional project-team effort.       |
| <b>Controlled improvement</b>        | Approved backlog items have owners, capacity, test evidence and release route.                                    |
| <b>Expansion assessment</b>          | New users, domains, questions or integrations have evidence, funding, readiness and governance approval.          |
| <b>Earlier-phase remediation</b>     | Structural issues are routed back to data readiness, governed foundation, architecture, validation or pilot work. |
| <b>Constraint or pause</b>           | Affected scope, users, communication, access and monitoring actions are defined.                                  |
| <b>Retirement or decommissioning</b> | Users, access, documentation, support, backlog, cost and audit evidence are closed or transferred cleanly.        |

The handover should make clear what remains live, what changes, what is constrained, who owns the next actions and what users or support teams need to know.



## 8. Key risks and failure modes

Phase 9 fails when the released service remains live but stops being actively controlled. The most common risks are not only technical outages. They are gradual loss of trust, silent scope expansion, unmanaged cost growth, stale data or semantic foundations, weak ownership and uncontrolled change.

| Risk / failure mode              | Why it matters                                                                       | Typical response                                                              |
|----------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Silent scope expansion</b>    | New users, questions, domains or workflows appear without approval.                  | Reinforce boundary control and route expansion through governance.            |
| <b>Weak operating ownership</b>  | The build team remains the informal support route.                                   | Confirm operating owner, support route and escalation model.                  |
| <b>Upstream drift</b>            | Data, model, metric, metadata or access changes alter live answers.                  | Maintain upstream change radar, BAU checks and regression triggers.           |
| <b>Trust degradation</b>         | Users receive plausible but wrong, unclear or poorly caveated answers.               | Review traces, caveats, answer quality and user interpretation.               |
| <b>Over-trust or misuse</b>      | Users rely on outputs outside the intended workflow or evidence boundary.            | Strengthen responsible-use guidance and user check-ins.                       |
| <b>Cost growth</b>               | Follow-up conversations, model calls, queries or support effort exceed expectations. | Monitor cost drivers and optimise or constrain usage.                         |
| <b>Feature-factory behaviour</b> | Every request becomes a backlog item for the current service.                        | Separate run, fix, optimise, improve, expand, constrain and defer.            |
| <b>Uncontrolled change</b>       | Prompt, model, routing, UI or validation changes alter behaviour without evidence.   | Use versioning, regression testing, release notes and rollback routes.        |
| <b>Support overload</b>          | The service becomes a helpdesk for every analytical question.                        | Clarify supported scope, improve guidance and route adjacent needs elsewhere. |
| <b>Stale documentation</b>       | Known limitations, runbooks or user guidance no longer match service behaviour.      | Keep documentation and guidance under BAU currency control.                   |
| <b>Misread adoption signals</b>  | High usage is treated as success, or low usage as failure, without context.          | Review usage with value, workflow fit, risk, cost and support evidence.       |
| <b>No clean stop route</b>       | The service continues because nobody can pause, retire or decommission it.           | Maintain lifecycle decision rights and decommissioning readiness.             |

### 8.1. Practitioner note

The main Phase 9 risk is gradual normalisation of exceptions. A manual access workaround, an unsupported question, an untested prompt change, a stale caveat or an informal support route may be acceptable once, under control. It becomes dangerous when it becomes the way the service is operated.